

T.C. Fırat Üniversitesi Mühendislik Fakültesi Yazılım Mühendisliği Bölümü

Veritabanı Yönetim Sistemleri Dersi Proje Raporu

Grup Numarası : 9

Takım Lideri ve Grup Üyelerinin Okul Numaraları, Ad ve Soyadları

- 240290020 - Uğur Erdoğan (Takım Lideri)
- 240290090 - Hilal Çoğul
- 250290096 - Ecenur Eke

Projenin Adı : Otel Otomasyonu

1. PROJE ÖZELLİKLERİ

1.1 Projenin Amacı

Bu projenin amacı; modern bir otel işletmesinin temel operasyonları (rezervasyon ekleme, boş oda kontrolü, otel doluluk kontrolü, müşterilerin ek hizmetlerden yararlanması ve yararlanılan hizmetlerin faturaya yansıtılması, fiyat güncellemelerinin yapılması) ilişkisel bir veritabanı mimarisi üzerinde güvenli bir şekilde optimize etmeyi, veri bütünlüğünü tetikleyiciler (triggers) ve saklı yordamlar (stored procedures) ile veritabanı seviyesinde koruyan bir otel otomasyon sistemi geliştirmektir. Proje, resepsiyonist ve admin rolleri arasında eş zamanlı, sıfır veri kaybı ve minimum gecikmeyle operasyonların yürütülmesini hedefler.

1.2. Veritabanı Bilgileri

Veritabanı Adı: otel_otomasyonu

Veritabanı Yönetim Sistemi: MySQL (İlişkisel veri modeli, View ve Stored Procedure destekleri için kullanılmıştır.)

Veritabanı Sürücüsü: mysql-connector-python (Python ile MySQL arasındaki veri iletişimini sağlamak için entegre edilmiştir.)

Tablo Yapısı ve İlişkisel Şema

1. Oda_Turleri: Odaların kategorilerini (Standart, Süit vb.) ve taban fiyatlarını tutar.
2. Musteriler: Otelde konaklayacak kişilerin temel iletişim ve kimlik bilgilerini barındırır.
3. Personeller: Sistemi (arayüzü) kullanacak otel çalışanlarının ve adminlerin giriş bilgilerini tutar.

4. Hizmetler: Otelin sunduğu ekstra ücretli hizmetlerin (SPA, Minibar vb.) tanımlandığı tablodur.
5. Odalar: Oteldeki fiziksel odaların numaralarını ve anlık durumlarını (Boş/Dolu) takip eder.
6. Rezervasyonlar: Sistemin ana merkezidir. Hangi müşterinin, hangi odada, hangi tarihlerde kalacağını birleştirir.
7. Rezervasyon_Hizmetleri: Bir rezervasyon süresince müşterinin kullandığı ekstra hizmetleri ve adetlerini kaydeder.
8. Faturalar: Rezervasyon bittiğinde kesilen toplam tutar ve ödeme yöntemini tutar.
9. Islem_Kayitlari: Sistemde hangi personelin ne zaman hangi işlemi yaptığını loglayan güvenlik tablosudur.

1.3. Backend için Kullanılan Programlama Dili, Web Framework Bilgileri

Programlama Dili: Python 3

Web Çerçevesi (Framework): FastAPI (Yüksek performanslı, asenkron ve RESTful API geliştirmeye uygun modern mimarisi nedeniyle tercih edilmiştir.)

Sunucu (ASGI Server): Uvicorn (FastAPI uygulamasını ayağa kaldırmak ve asenkron istekleri yönetmek için kullanılmıştır.)

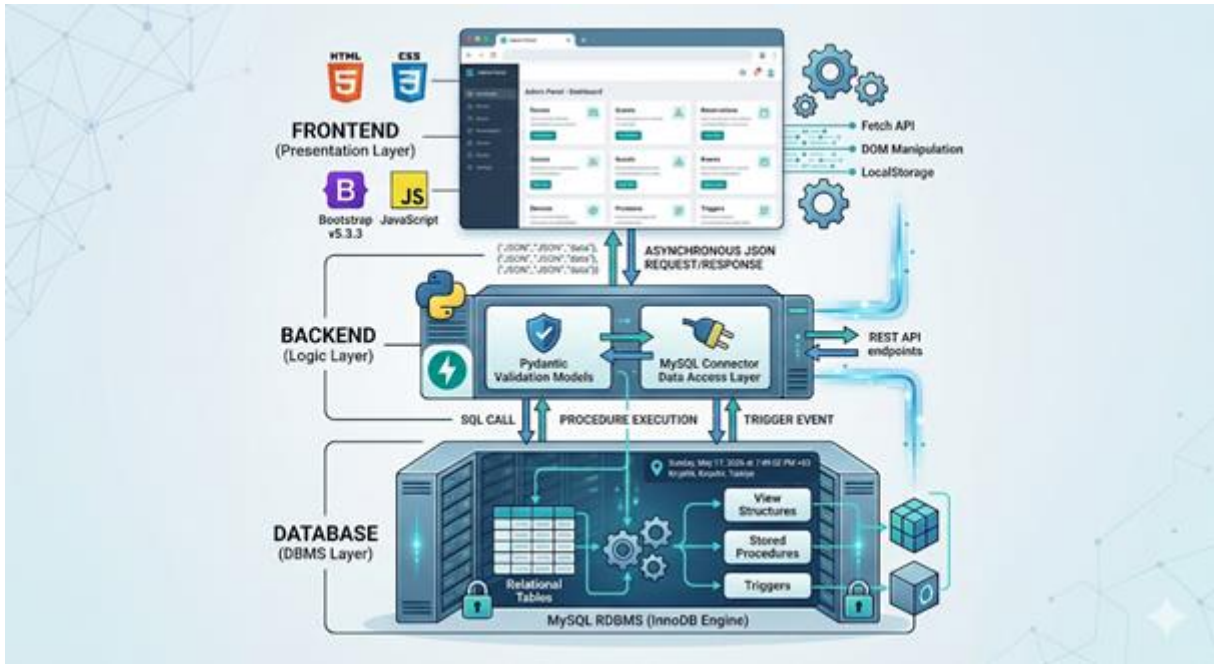
1.4. Frontend için kullanılan araçlar

İşaretleme ve Şekillendirme: HTML5 & CSS3 (Modern, anlamsal ve erişilebilir arayüz yapısı için.)

Programlama Dili: JavaScript (ES6+) (Dinamik sayfa etkileşimleri, asenkron veri akışı ve tarayıcı tabanlı iş mantığı için.)

CSS Çerçevesi (Framework): Bootstrap 5.3 (Grid sistemi, mobil uyumluluk (responsive) ve hazır UI bileşenleri (Modallar, tablolar) için kullanılmıştır.)

1.5. Proje Mimarisinin Gösterimi



Şekil 1: Otel Otomasyon Sistemi Üç Katmanlı (Full-Stack) Mimari ve Veri Akış Şeması

Sistemimiz, modern yazılım mühendisliği standartlarına uygun olarak Üç Katmanlı Mimari (3-Tier Architecture) prensibiyle tasarlanmıştır. Bu yapı; kullanıcı arayüzünü sunan Sunum Katmanı (Frontend), iş kurallarını ve veri doğrulamasını yöneten Mantık Katmanı (Backend) ve verilerin güvenli bir şekilde saklanıp işlendiği Veri Katmanı (Database) olmak üzere üç bağımsız birimden oluşmaktadır.

Frontend tarafında HTML5, CSS3 ve modern JavaScript (ES6+) kullanılarak oluşturulan istemci, REST tabanlı FastAPI backend sunucusuyla asenkron Fetch API üzerinden JSON formatında haberleşmektedir. Mantık katmanında yer alan Python/FastAPI uygulaması, Pydantic modelleriyle veri güvenliğini sağlarken, mysql-connector-python sürücüsü aracılığıyla MySQL veritabanına bağlanmaktadır. Veritabanı katmanı ise yalnızca bir veri deposu olarak değil; içerdiği tetikleyiciler (triggers) ve saklı yordamlar (stored procedures) sayesinde iş mantığının bir kısmını kendi üzerinde çalıştıran aktif bir mimari elemanı olarak görev yapmaktadır. Bu mimari izolasyon, projenin modülerliğini ve sürdürülebilirliğini maksimum düzeye çıkarmaktadır.

1.6. Projenin Klasör Yapısı

OTEL-OTOMASYONU/

```
|
|
| └── backend/                                # Mantık ve API Katmanı
|   | └── __pycache__/                        # Python derlenmiş önbellek dosyaları
|   | └── venv/                               # Python sanal ortam (virtual environment) klasörü
|   | └── .gitignore                         # Git versiyon kontrolü hariç tutma listesi
|   | └── .gitkeep                           # Boş klasör tutucu dosya
|   | └── database.py                        # MySQL Bağlantı ve Pool Yönetimi
|   | └── main.py                            # FastAPI Uygulama ve Endpoint Tanımları
|   | └── queries.py                         # Ham SQL Sorguları ve View Referansları
|   | └── requirements.txt                   # Proje bağımlılıkları (pip kütüphaneleri)
|
| └── database/                              # VTYS ve Şema Katmanı (SQL Scriptleri)
|   | └── 01_tablo_kurulumu.sql
|   | └── 02_ornek_veriler.sql
|   | └── 03a_tetikleyici_oda_durumu_guncelle.sql
|   | └── 03b_tetikleyici_tarih_kontrolu.sql
|   | └── 04a_sanalTablo_aktif_musteriler.sql
|   | └── 04b_sanalTablo_fatura_bekleyenler.sql
|   | └── 04c_sanalTablo_vw_dashboard_ozet.sql
|   | └── 04d_sanalTablo_vw_rezervasyon_listesi.sql
|   | └── 04e_sanalTablo_vw_oda_detaylari.sql
|   | └── 04f_sanalTablo_vw_aktif_borclar.sql
|   | └── 05a_sanalMethod_fatura_hesapla.sql
|   | └── 05b_sanalMethod_hizmet_ekleme.sql
|   | └── 05c_sanalMethod_yeni_rezervasyon_ekle.sql
|   | └── 05d_sanalMethod_musait_oda_Ara.sql
|   | └── 05e_sanalMethod_rezervasyon_durum_guncelle.sql
|   | └── 05f_sanalMethod_oda_turu_fiyat_guncelle.sql
|   | └── 05g_sanalMethod_hizmet_fiyat_guncelle.sql
|   | └── 99_tam_sifirlama.sql
|
|
| └── docs/                                  # Dokümantasyon Klasörü
|   | └── Veritabanı_Yönetim_Sistemleri_Dersi_Proje_Raporu.docx
```

```

|
|— frontend/                                     # Sunum Katmanı (Kullanıcı Arayüzü)
|   |— css/
|       |— styles.css                         # Ortak CSS Stil Dosyası
|   |— img/                                  # Kullanılan Fotoğrafların Olduğu Klasör
|   |— js/                                   # JavaScript Mantık ve API Dosyaları
|       |— api.js
|       |— ayarlar.js
|       |— common.js
|       |— dashboard.js
|       |— gecmis.js
|       |— islemler.js
|       |— misafirler.js
|       |— odalar.js
|       |— rezervasyonlar.js
|       |— yetkili-auth.js
|   |
|   |— yetkili/                               # Özel Yetkili Yönlendirme Klasörü
|       |— index.html                       # Yetkili Giriş/Karşılama Sayfası
|   |— admin.html
|   |— ayarlar.html
|   |— dashboard.html
|   |— gecmis.html
|   |— islemler.html
|   |— misafirler.html
|   |— odalar.html
|   |— rezervasyonlar.html
|   |— yetkili.html
|
|— README.md                                  # Proje genel kurulum ve çalıştırma talimatları

```

1.7. Proje Modülleri

Sistemimiz, karmaşık otel operasyonlarını yönetebilmek için birbirleriyle entegre çalışan 6 ana modülden oluşmaktadır. Her bir modül, frontend arayüzünden başlayıp FastAPI backend mantığına ve MySQL veritabanındaki saklı yordam/tetikleyici yapılarına kadar uzanan tam yığın (full-stack) bir döngüyü kapsar.

1. Akıllı Rezervasyon ve Müşteri Yönetim Modülü

- **İşlevi:** Otel müşterilerinin sisteme kaydı, müsaitlik sorgulanması ve rezervasyon atamalarının yapıldığı ana operasyon modülüdür.
- **Teknik Altyapısı:** Veritabanında sp_YeniRezervasyonEkle saklı yordamı ile çalışır. T.C. kimlik numarası üzerinden müşterinin daha önce otelde kalıp kalmadığını kontrol eder. Yeni müşteriye otomatik kaydeder, ardından istenen tarihlerde çakışmayan ve "Arızalı" olmayan uygun odayı akıllı algoritmasıyla bularak atar. Ayrıca trg_tarih_kontrol tetikleyicisi sayesinde geçmiş tarihe veya hatalı aralıklara rezervasyon girilmesi veritabanı seviyesinde reddedilir.

2. Oda Durumu ve Kat Operasyonları Modülü

- **İşlevi:** Oteldeki tüm odaların (Boş, Dolu, Temizlikte, Arızalı) durumlarını kat planına göre gerçek zamanlı ve görsel olarak takip etmeyi sağlar.
- **Teknik Altyapısı:** Frontend tarafında odalar.html arayüzünde renkli bir matris olarak çalışır. Veritabanında vw_oda_detaylari sanal tablosundan beslenir. trg_oda_durumu_guncelle tetikleyicisi sayesinde, bir rezervasyon onaylandığında oda otomatik "Dolu", müşteri çıkış yaptığı anda ise "Temizlikte" durumuna geçer. Temizlik personeli odayı temizlediğinde tek tıkla oda tekrar "Boş" durumuna çekilir.

3. Finans, Fatura ve Check-out (Çıkış) Modülü

- **İşlevi:** Müşterinin konakladığı gün sayısı ve kaldığı süre boyunca aldığı tüm ekstra hizmetleri toplayarak nihai faturayı oluşturur ve tahsilat/çıkış işlemlerini kapatır.
- **Teknik Altyapısı:** islemler.html üzerinden yönetilir. Çıkış anında sp_FaturaKes prosedürü devreye girer. Bu prosedür DATEDIFF fonksiyonu ile giriş-çıkış tarihleri arasındaki gün sayısını bularak oda ücretini hesaplar, ardından Rezervasyon_Hizmetleri tablosundaki kayıtları ekleyerek genel toplamı oluşturur ve faturayı keser. Faturası kesilmeyen ve güncel borç kaydı bulunan müşteriler vw_aktif_borclar sanal tablosu (view) üzerinden dinamik olarak sorgulanarak sistemde bekleyen ödemeler listesinde gösterilir.

4. Ekstra Hizmetler ve Dinamik Fiyatlandırma Modülü

- **İşlevi:** Minibar, SPA, VIP transfer gibi ek hizmetlerin misafirlerin hesaplarına anlık işlenmesini ve otel yönetiminin (Admin) oda/hizmet fiyatlarını dinamik olarak güncellemesini sağlar.
- **Teknik Altyapısı:** Misafire hizmet eklenirken sp_HizmetEkle prosedürü çalışır. Yönetici fiyat güncellemek istediğinde ise sp_OdaTuruFiyatGuncelle ve sp_HizmetFiyatGuncelle yordamları ile veritabanı tabloları anında güncellenir.

5. Dashboard (Pano) ve Sistem Loglama Modülü

- **İşlevi:** Otel yöneticisine o günkü aktif misafir sayısını, kasadaki toplam ciroyu ve beklenen çıkışları tek bir operasyon merkezinde sunar. Ayrıca sistemde yapılan tüm işlemleri denetim amacıyla kayıt altına alır.
- **Teknik Altyapısı:** Sistemde oda durumu değişmesi, rezervasyon onaylanması veya fatura kesilmesi gibi tüm kritik işlemler, istemci tarafında yüksek performanslı ve anlık takibe izin veren localStorage loglama yapısıyla asenkron olarak yönetilmekte ve gecmis.html üzerinden kronolojik olarak izlenebilmektedir.

6. Kimlik Doğrulama (Auth) ve Yetkilendirme Modülü

- **İşlevi:** Müşteri paneli ile personel yönetim panelini ayırarak, sisteme sadece yetkili personelin (Admin, Resepsiyonist vb.) güvenli bir şekilde erişmesini sağlar.
- **Teknik Altyapısı:** Personeller tablosundaki kimlik bilgileri, FastAPI backend'indeki /login uç noktasından doğrulanır. Başarılı bir kimlik doğrulaması sırasında istemci tarafında session kontrolü amacıyla localStorage üzerinde bir oturum token'ı oluşturulur. Yetkisiz erişimler tarayıcı tabanlı yönlendirme mantığı ile kontrol edilerek yönetim paneli güvenliği simüle edilir.

2. PROJE YÖNETİMİ

2.1.Takım Üyeleri Tanımı

Üye No	Ad-Soyad	Öğrenci No	Görevi (Takım Lideri/Takım Üyesi)
1	Uğur Erdoğan	240290020	Takım Lideri / Backend Geliştirici & API Tasarımcısı
2	Hilal Çoğul	240290090	Takım Üyesi / Veritabanı Yöneticisi (DBA) & SQL Mimarı

3	Ecenur Eke	250290096	Takım Üyesi / UI/UX Tasarımcısı & Frontend Geliştirici
---	------------	-----------	--

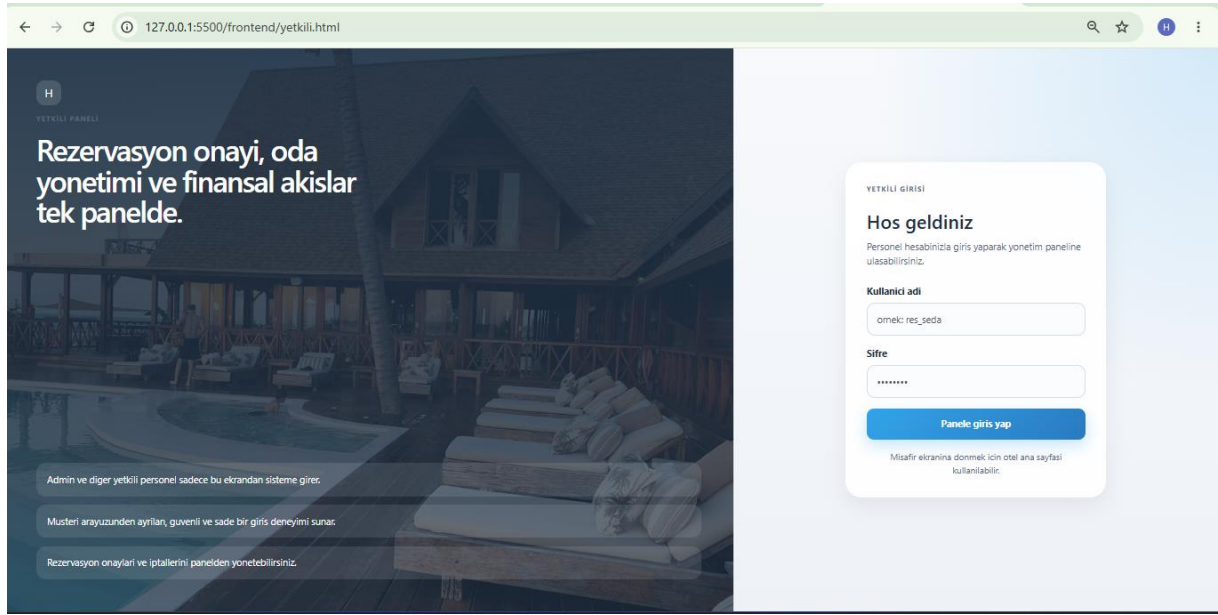
2.2.Görev Dağılımı

İş Paketi No	İş Paketinin Adı ve Hedefleri	Görevlendirilen Takım Üyesi
WP1	Veritabanı Şema Tasarımı ve Veri Bütünlüğü (DDL): Rezervasyon, Müşteri, Finans ve Oda Yönetimi modüllerinin altyapısını oluşturan ilişkisel tabloların (1NF, 2NF, 3NF standartlarında) tasarlanması ve PK/FK/Unique kısıtlamalarının sisteme entegrasyonu.	Hilal Çoğul
WP2	Gelişmiş VTYS Programlama ve İş Kuralları (Programmable SQL): Akıllı Rezervasyon Modülü (sp_YeniRezervasyonEkle), Finans Modülü (sp_FaturaKes) ve Oda Operasyonları Modülü (trg_oda_durumu_guncelle) için gerekli saklı yordam, tetikleyici ve sanal tabloların (view) kodlanması.	Hilal Çoğul
WP3	Backend REST API Altyapısı ve Güvenlik: FastAPI web framework'ü kullanılarak projenin sunucu iskeletinin kurulması, MySQL bağlantı havuzunun oluşturulması, Kimlik Doğrulama Modülü ve veri doğrulama (Pydantic) modellerinin yapılandırılması.	Uğur Erdoğan

WP4	API İş Mantığı (Endpoint) Entegrasyonu: Pano (Dashboard), Ekstra Hizmetler ve Rezervasyon modüllerinin veritabanı yordamlarıyla haberleşmesini sağlayan asenkron API uç noktalarının (endpoints) geliştirilmesi ve HTTP hata yönetiminin (exception handling) sağlanması.	Uğur Erdoğan
WP5	Arayüz Tasarımı (UI/UX) ve Görsel İskelet: Operasyon Panosu (Dashboard), Oda Durum Takip, Rezervasyon ve Fatura ekranlarının Bootstrap v5 kütüphanesi ile modern, mobil uyumlu (responsive) ve kullanıcı dostu bir yapıda (HTML/CSS) tasarlanması.	Ecenur Eke
WP6	Asenkron İstemci (Frontend) Geliştirme ve API Haberleşmesi: Arayüz tasarımlarının fetch API ile backend uç noktalarına bağlanması; rezervasyon oluşturma, fatura kesme, durum güncelleme gibi modül işlemlerinin JavaScript (DOM manipülasyonu) ile dinamik hale getirilmesi.	Ecenur Eke

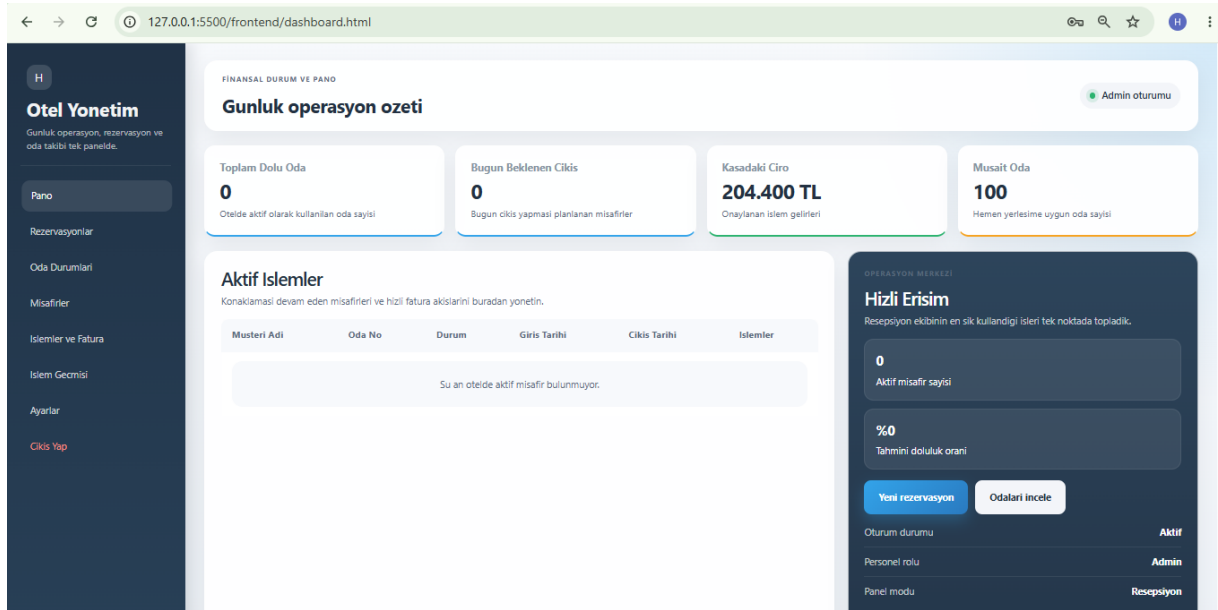
3. SONUÇLAR

3.1.Ekran Çıktıları



Şekil 3.1: Yetkili Giriş (Login) Ekranı

İlgili Bileşen: Kimlik Doğrulama (Auth) ve Yetkilendirme Modülü. Personel kimlik bilgilerinin backend katmanında doğrulanarak istemci tarafında session tabanlı geçici token (localStorage) kontrolünün simüle edildiği ve yetkisiz panel erişimlerinin tarayıcı tabanlı yönlendirme mantığıyla engellendiği personel giriş arayüzüdür.



Şekil 3.2: Yönetim Paneli (Dashboard) ve Operasyon Özeti

İlgili Bileşen: Dashboard (Pano) Modülü. vw_dashboard_ozet sanal tablosundan beslenerek otelin anlık doluluk oranını, aktif misafir sayısını ve kasadaki toplam ciroyu tek ekranda sunan kontrol merkezidir.

REZERVASYON YÖNETİMİ

Yeni kayıt ve aktif rezervasyonlar

Admin oturumu

Yeni rezervasyon

Backend üzerinden doğrudan yeni rezervasyon oluşturun.

TC Kimlik No:

Oda Tipi:

Ad:

Soyad:

Telefon:

E-posta:

Giriş Tarihi:

Çıkış Tarihi:

Rezervasyonu kaydet

Ödeme Durumu

Backend tarafında ödemesi bekleyen kayıtlar burada listelenir.

Su an bekleyen ödeme kaydı görünmüyor.

Tüm rezervasyonlar

Admin ve resepsiyon personeli burada onay, iptal ve durum takibini yapabilir.

Rez. ID	Müşteri	Oda	Oda Tipi	Giriş	Çıkış	Durum	Ödeme	İşlem
---------	---------	-----	----------	-------	-------	-------	-------	-------

Şekil 3.3: İç Sistem Yeni Rezervasyon ve Liste Ekranı

İlgili Bileşen: Akıllı Rezervasyon ve Müşteri Yönetim Modülü. Resepsiyon personelinin sp_YeniRezervasyonEkle prosedürünü kullanarak çakışmasız oda ataması yaptığı ve rezervasyonları onayladığı ekrandır.

ODA DURUMLARI

Kat ve oda takibi

Admin oturumu

Tüm odalar

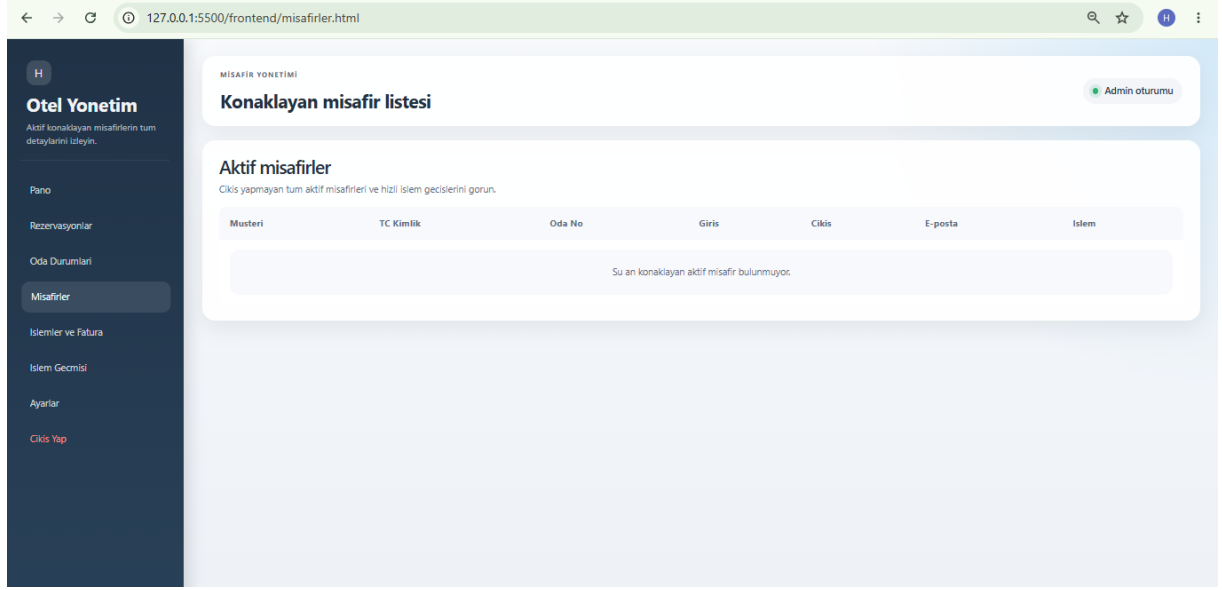
Boş, dolu, temizlikte ve arızalı odalar renkli kartlarla gösterilir.

Boş Temizlikte Dolu

101 Ekonomik Oda Boş	106 Ekonomik Oda Boş	204 Ekonomik Oda Boş	209 Ekonomik Oda Boş
303 Ekonomik Oda Boş	307 Ekonomik Oda Boş	403 Ekonomik Oda Boş	407 Ekonomik Oda Boş
410 Ekonomik Oda Boş	501 Ekonomik Oda Boş	505 Ekonomik Oda Boş	509 Ekonomik Oda Boş

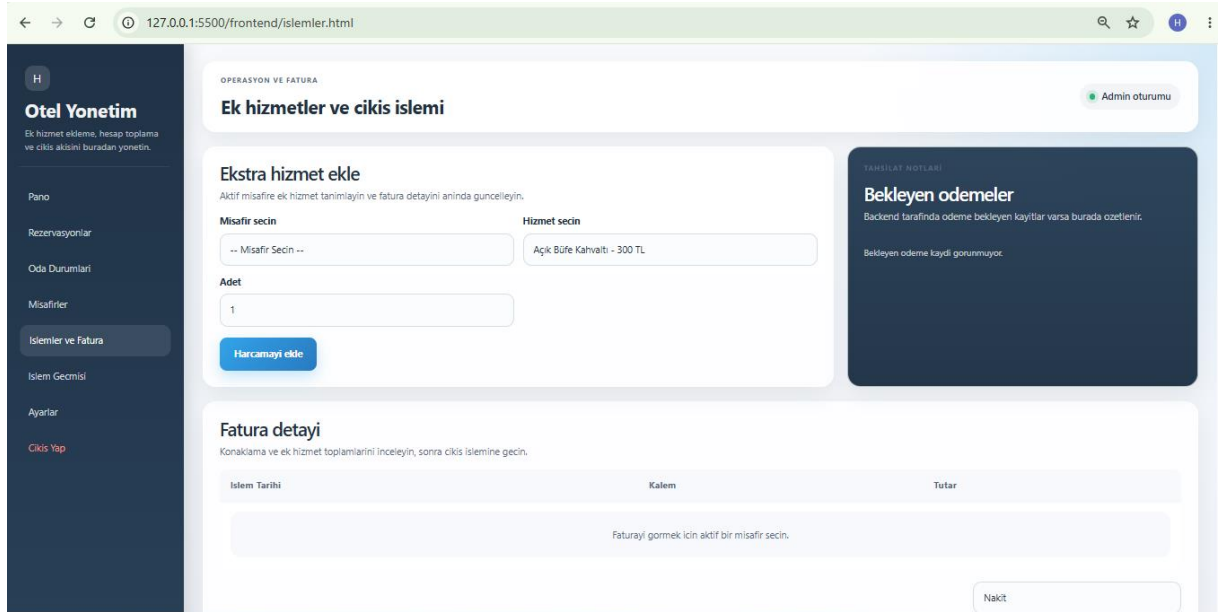
Şekil 3.4: Dinamik Oda Durumları (Kat Planı) Ekranı

İlgili Bileşen: Oda Durumu ve Kat Operasyonları Modülü. Oteldeki tüm odaların anlık durumlarını renkli matris mimarisıyla gösteren ve temizlik işlemlerinin yönetildiği arayüzdür.



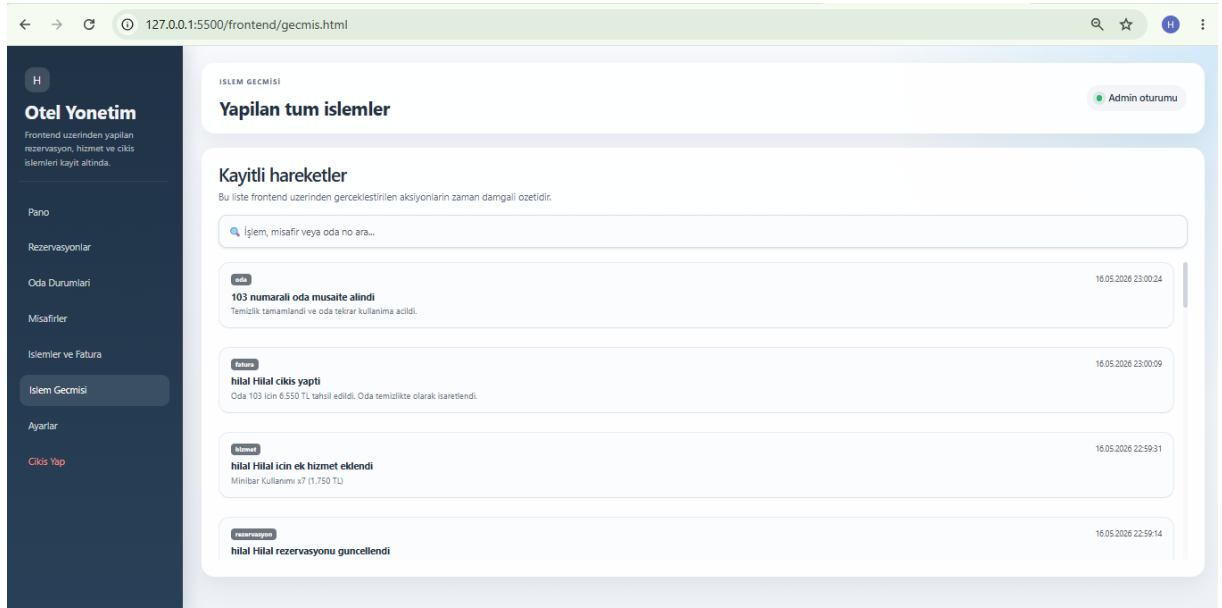
Şekil 3.5: Aktif Konaklayan Misafirler Ekranı

İlgili Bileşen: Müşteri Yönetim Modülü. vw_aktif_musteriler sanal tablosu aracılığıyla o an otelde konaklayan müşterilerin bilgilerinin listelendiği yönetim arayüzüdür.



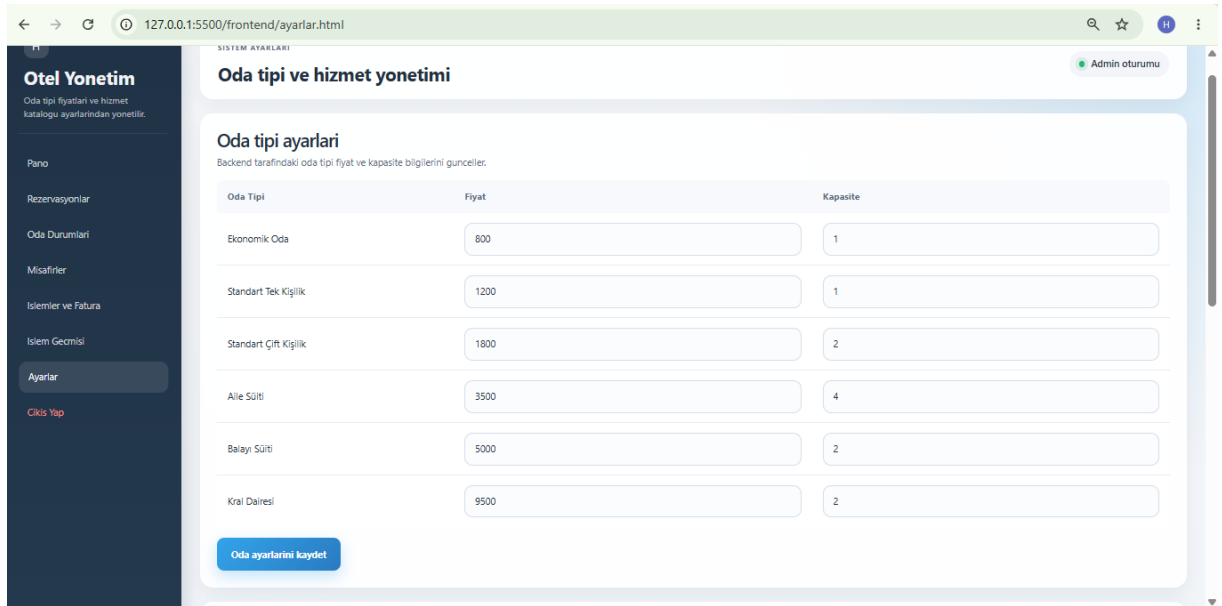
Şekil 3.6: İşlemler, Ekstra Hizmetler ve Fatura Ekranı

İlgili Bileşen: Finans, Fatura ve Check-out Modülü. Konaklayan misafirlere ekstra hizmetlerin eklendiği ve çıkış anında sp_FaturaKes prosedürü tetiklenerek nihai hesabın kapatıldığı ödeme ekranıdır.



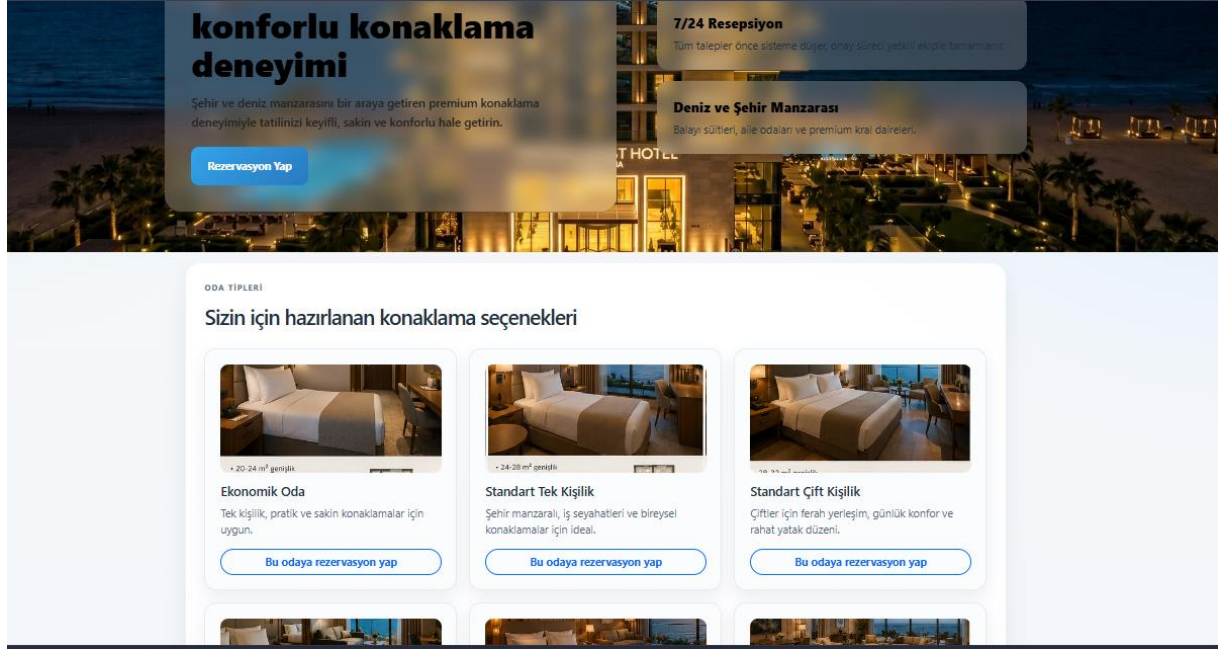
Şekil 3.7: Sistem İşlem Geçmişi (Log) Ekranı

İlgili Bileşen: Sistem Loglama Modülü. Arayüz üzerinden gerçekleştirilen tüm operasyonların ve durum güncellemelerinin, istemci tarafında (localStorage) anlık kronolojik log yapısıyla kayıt altına alınarak asenkron olarak listelendiği denetim ekranıdır.



Şekil 3.8: Dinamik Fiyatlandırma ve Sistem Ayarları Ekranı

İlgili Bileşen: Ekstra Hizmetler ve Dinamik Fiyatlandırma Modülü. Yönetici (Admin) yetkisiyle oteldeki oda türlerinin taban fiyatlarının ve ekstra hizmet ücretlerinin (SPA, Minibar vb.), veritabanındaki sp_OdaTuruFiyatGuncelle ve sp_HizmetFiyatGuncelle saklı yordamları tetiklenerek anında güncellendiği yönetim arayüzüdür.



Şekil 3.9: Müşteri Karşılama (Landing Page) ve Oda Tanıtım Ekranı

İlgili Bileşen: Otel müşterilerinin sisteme ilk erişim sağladığı; otel konseptinin, oda türlerinin ve kapasite standartlarının modern ve anlamsal HTML5/CSS3 mimarisiyle statik olarak sergilendiği ana vitrin arayüzüdür.

3.2. Program Kodları

Projenin kodlarının bulunduğu Drive linki:

https://drive.google.com/drive/folders/1x19OqUvVCqtMFPkJ4YexgtCznZencJVH?usp=drive_link

Bu bölümde, otel otomasyon sisteminin mantık katmanını (Logic Layer) oluşturan FastAPI asenkron uç noktalarından (endpoints) kritik olanları, ilişkili veritabanı yordamları ve veri akış mimarisiyle birlikte sunulmuştur.

A. Gösterge Paneli ve Merkezi Operasyon Özeti API Entegrasyonu

Kontrol paneli (Dashboard), ağ trafiğini ve veritabanı üzerindeki sorgu yükünü minimize etmek amacıyla tek bir uç nokta üzerinden çoklu veri kümesi dönmektedir. Sistem, anlık doluluk durumunu ve boş oda sayılarını dinamik olarak hesaplarken, ciro bilgilerini ilgili View yapısı üzerinden asenkron olarak birleştirir. Ayrıca CURDATE() fonksiyonuyla entegre çalışan bir alt sorgu sayesinde bugün çıkış yapması beklenen misafir sayısını anlık takip eder.

```
@app.get("/dashboard/ozet")
def dashboard_verilerini_getir():
    conn = get_db_connection()
    if not conn:
```

```

        return {"dolu_oda_sayisi": 0, "bos_oda_sayisi": 0, "toplaml_ciro": 0,
"bugun_cikis_yapacaklar": 0}

    try:
        cursor = conn.cursor(dictionary=True)

        # 1. Dolu ve Boş Oda Sayısını Bulma
        cursor.execute(queries.GET_AKTIF_MUSTERILER)
        aktif_misafirler = cursor.fetchall()
        gercek_dolu_oda = len(aktif_misafirler)

        cursor.execute("SELECT COUNT(*) as toplam FROM Odalar")
        toplam_oda_sonuc = cursor.fetchone()
        toplam_oda = toplam_oda_sonuc["toplam"] if toplam_oda_sonuc else 0
        gercek_bos_oda = toplam_oda - gercek_dolu_oda

        # 2. Ciro ve Özet Bilgisi
        cursor.execute(queries.GET_DASHBOARD_OZET)
        ozet = cursor.fetchone()

        # Bugün Çıkış Yapacak Misafir Sayısı
        # (Otomatik olarak bugünün tarihini kontrol eder)
        bugun_sorgusu = """
            SELECT COUNT(*) as cikis_sayisi
            FROM Rezervasyonlar
            WHERE rezerve_durumu NOT IN ('İptal Edildi', 'Tamamlandı')
            AND rezerve_cikis_tarihi = CURDATE()
        """
        cursor.execute(bugun_sorgusu)
        bugun_cikis_sonuc = cursor.fetchone()
        bugun_cikis = bugun_cikis_sonuc["cikis_sayisi"] if bugun_cikis_sonuc else 0

        # 4. Frontend'e gidecek standart paket
        sonuc = {
            "dolu_oda_sayisi": gercek_dolu_oda,
            "bos_oda_sayisi": gercek_bos_oda,
            "bugun_cikis_yapacaklar": bugun_cikis,
            "toplaml_ciro": 0.0
        }

        # SQL'den dönen ciro kolonunun adı ne olursa olsun yakalıyoruz
        if ozet:
            ciro_degeri = ozet.get("toplaml_ciro") or ozet.get("ciro") or ozet.get("toplaml_tutar") or 0
            sonuc["toplaml_ciro"] = float(ciro_degeri)

        return sonuc

    except Exception as e:

```

```

print("Dashboard Hatası:", str(e))
return {"dolus_oda_sayısı": 0, "bos_oda_sayısı": 0, "toplam_ciro": 0,
"bugun_cikis_yapacaklar": 0}
finally:
    if conn and conn.is_connected():
        cursor.close()
        conn.close()

```

B. Dinamik Oda Türü Yapılandırması ve Fiyat-Kapasite Senkronizasyonu

Yönetici panelinden tetiklenen oda türü güncellenmesi, veritabanı katmanındaki ilişkisel şemanın korunması adına doğrudan sp_OdaTuruGuncelle saklı yordamı (Stored Procedure) çağrılarak yürütülür. Bu uç nokta; oda türü adını, yeni fiyat politikasını ve güncellenen maksimum kapasite kriterlerini tek bir işlem kümesiyle veritabanına parametrik olarak iletir.

```

@app.put("/ayarlar/oda-fiyat")
def oda_fiyati_guncelle(veri: FiyatGuncelleme):
    conn = get_db_connection()
    try:
        cursor = conn.cursor()
        cursor.callproc("sp_OdaTuruGuncelle", (veri.oda_turu_adi, veri.yeni_fiyat,
veri.yeni_kapasite))
        conn.commit()
        return {"mesaj": "Oda turu fiyatı başarıyla guncellendi."}
    finally:
        conn.close()

```

C. Finans Katmanı ve Aktif Borç Takip Mekanizması

Sistemde faturalandırılmayı bekleyen veya ara harcama dökümleri incelenmek istenen misafirlerin finansal kayıtları, veritabanı seviyesinde optimize edilmiş olan vw_aktif_borclar sanal tablosu (View) üzerinden çekilir. Bu asenkron metot, fatura kesim aşaması öncesinde veritabanı verilerinin güncel halini arayüze aktarır.

```

@app.get("/finans/odeme-bekleyenler")
def odeme_bekleyenleri_getir():
    conn = get_db_connection()
    if not conn:
        return []

    try:
        cursor = conn.cursor(dictionary=True)
        # Artık veritabanındaki hazır View'ı çağırıyoruz
        cursor.execute(queries.GET_AKTIF_BORCLAR)
        return cursor.fetchall()
    except Exception as e:

```



```

print("Bekleyen Ödeme Hatası:", str(e))
return []
finally:
    if conn and conn.is_connected():
        cursor.close()
        conn.close()

```

D. Personel Oturum Yönetimi ve Kimlik Doğrulama Altyapısı

Sisteme erişmek isteyen personelin yetkileri ve rolleri, veritabanındaki Personeller tablosundaki kayıtlar üzerinden filtrelendir. Başarılı giriş işlemlerinde backend katmanı, tarayıcı tabanlı yönlendirme kurallarını ve oturum sürekliliğini istemci katmanında (localStorage) güvenli bir şekilde simüle etmek amacıyla benzersiz bir session token mimarisi döndürür.

```

@app.post("/login")
def sisteme_giris_yap(bilgiler: PersonelGiris):
    conn = get_db_connection()
    if not conn:
        raise HTTPException(status_code=500, detail="Veritabani baglantisi kurulamadi")

    try:
        cursor = conn.cursor(dictionary=True)
        cursor.execute(queries.GET_PERSONEL_GIRIS, (bilgiler.kullanici_adi, bilgiler.sifre))
        personel = cursor.fetchone()
        if personel:
            return {
                "mesaj": "Giris basarili",
                "token": f"fake-jwt-token-{personel['personel_id']}",
                "role": personel["personel_rol"],
            }

        raise HTTPException(status_code=401, detail="Gecersiz kullanıcı adı veya şifre")
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
    finally:
        if conn and conn.is_connected():
            cursor.close()
            conn.close()

```

E. Rezervasyon Bazlı Ekstra Hizmet Tüketimlerinin İlişkisel Listelenmesi

Konaklayan misafirlerin kaldıkları süre boyunca yararlandıkları ekstra harcamalar (SPA, Minibar vb.), rezervasyon_hizmetleri ara tablosu ile hizmetler ana tablosunun dinamik bir şekilde JOIN edilmesiyle listelenir. Bu uç nokta, her bir kalemin adedini ve birim fiyatını anlık ilişkisel modelden çekerek doğru hesap dökümünün oluşturulmasını sağlar.

```

@app.get("/rezervasyonlar/{rezervasyon_id}/hizmetler")

```

```

def rezervasyon_hizmetlerini_getir(rezervasyon_id: int):
    conn = get_db_connection()
    try:
        cursor = conn.cursor(dictionary=True)
        sorgu = """
            SELECT rh.hizmet_id, h.hizmet_adi, h.hizmet_birim_fiyat, rh.hizmet_adet
            FROM rezervasyon_hizmetleri rh
            JOIN hizmetler h ON rh.hizmet_id = h.hizmet_id
            WHERE rh.rezervasyon_id = %s
        """
        cursor.execute(sorgu, (rezervasyon_id,))
        return cursor.fetchall()
    finally:
        conn.close()

```

F. Veritabanı Bağlantı Yönetimi (database.py)

Sistemin MySQL veritabanı ile kesintisiz iletişim kurmasını sağlayan temel bağlantı modülüdür. İşletim sistemi seviyesindeki olası IPv6 (localhost) DNS çözümleme çakışmalarını önlemek ve ağ gecikmesini düşürmek adına, bağlantı doğrudan IPv4 (127.0.0.1) protokolü üzerinden sağlanmıştır. Ayrıca bağlantı konfigürasyonuna eklenen use_pure=True parametresi sayesinde, veritabanı sürücüsünün platform bağımsız (pure Python) çalışması zorlanarak olası C eklentisi (C-extension) derleme hatalarının önüne geçilmiştir. Veritabanı erişiminde yaşanabilecek olası kesintiler, hata yakalama (Try-Except) blokları ile kontrol altına alınmıştır.

```

import mysql.connector
from mysql.connector import Error

def get_db_connection():
    try:
        connection = mysql.connector.connect(
            host='127.0.0.1',
            database='otel_otomasyonu',
            user='root',
            password='',
            use_pure=True
        )
        if connection.is_connected():
            return connection
    except Error as e:
        print(f"Veritabanı bağlantı hatası: {e}")
        return None

```

3.3. Sorgular

Bu bölüm, projenin temel SQL programlama yapılarını içerir.

- 01_tablo_kurulumu.sql : Veritabanı tablolarının kurulması için gereken sql sorgularını barındırır.

```
• CREATE DATABASE IF NOT EXISTS otel_otomasyonu CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
• USE otel_otomasyonu;
•
• CREATE TABLE Oda_Turleri (
•     odaTur_id INT AUTO_INCREMENT PRIMARY KEY,
•     odaTur_adi VARCHAR(50) NOT NULL,
•     odaTur_kapasite INT NOT NULL,
•     odaTur_taban_fiyat DECIMAL(10, 2) NOT NULL,
•     odaTur_aciklama TEXT
• );
•
• CREATE TABLE Musteriler (
•     muster_i_id INT AUTO_INCREMENT PRIMARY KEY,
•     muster_i_adi VARCHAR(50) NOT NULL,
•     muster_i_soyadi VARCHAR(50) NOT NULL,
•     muster_i_tc_no CHAR(11) UNIQUE NOT NULL,
•     muster_i_telefon VARCHAR(15),
•     muster_i_email VARCHAR(100)
• );
•
• CREATE TABLE Personeller (
•     personel_id INT AUTO_INCREMENT PRIMARY KEY,
•     personel_adi VARCHAR(50) NOT NULL,
•     personel_soyadi VARCHAR(50) NOT NULL,
•     personel_rol ENUM('Admin', 'Resepsiyonist', 'Temizlik', 'Mutfak')
  DEFAULT 'Resepsiyonist',
•     personel_kullanici_adi VARCHAR(30) UNIQUE NOT NULL,
•     personel_sifre VARCHAR(255) NOT NULL
• );
•
• CREATE TABLE Hizmetler (
•     hizmet_id INT AUTO_INCREMENT PRIMARY KEY,
•     hizmet_adi VARCHAR(100) NOT NULL,
•     hizmet_birim_fiyat DECIMAL(10, 2) NOT NULL
• );
•
• CREATE TABLE Odalar (
•     oda_id INT AUTO_INCREMENT PRIMARY KEY,
•     oda_no VARCHAR(10) UNIQUE NOT NULL,
•     oda_kat INT,
```

```

•     odaTur_id INT,
•     oda_durumu ENUM('Boş', 'Dolu', 'Temizlikte', 'Arızalı') DEFAULT
'Boş',
•     CONSTRAINT fk_oda_odaTuru FOREIGN KEY (odaTur_id) REFERENCES
Oda_Turleri(odaTur_id) ON DELETE SET NULL
• );
•
• CREATE TABLE Rezervasyonlar (
•     rezervasyon_id INT AUTO_INCREMENT PRIMARY KEY,
•     musteri_id INT,
•     oda_id INT,
•     rezerve_giris_tarihi DATE NOT NULL,
•     rezerve_cikis_tarihi DATE NOT NULL,
•     rezerve_durumu ENUM('Beklemede', 'Onaylandı', 'Tamamlandı', 'İptal
Edildi') DEFAULT 'Beklemede',
•     CONSTRAINT fk_rez_musteri FOREIGN KEY (musteri_id) REFERENCES
Musteriler(musteri_id) ON DELETE CASCADE,
•     CONSTRAINT fk_rez_oda FOREIGN KEY (oda_id) REFERENCES Odalar(oda_id)
ON DELETE CASCADE
• );
•
• CREATE TABLE Rezervasyon_Hizmetleri (
•     rezervasyon_hizmet_id INT AUTO_INCREMENT PRIMARY KEY,
•     rezervasyon_id INT,
•     hizmet_id INT,
•     hizmet_adet INT,
•     hizmet_islem_tarihi DATETIME DEFAULT CURRENT_TIMESTAMP,
•     CONSTRAINT fk_hizmet_rez FOREIGN KEY (rezervasyon_id) REFERENCES
Rezervasyonlar(rezervasyon_id) ON DELETE CASCADE,
•     CONSTRAINT fk_hizmet_tip FOREIGN KEY (hizmet_id) REFERENCES
Hizmetler(hizmet_id)
• );
•
• CREATE TABLE Faturalar (
•     fatura_id INT AUTO_INCREMENT PRIMARY KEY,
•     rezervasyon_id INT UNIQUE,
•     fatura_toplam_tutar DECIMAL(10, 2) NOT NULL,
•     fatura_odeme_tarihi DATETIME DEFAULT CURRENT_TIMESTAMP,
•     fatura_odeme_yontemi ENUM('Nakit', 'Kredi Kartı') NOT NULL,
•     CONSTRAINT fk_fatura_rez FOREIGN KEY (rezervasyon_id) REFERENCES
Rezervasyonlar(rezervasyon_id)
• );
•
• CREATE TABLE Islem_Kayitlari (
•     kayit_id INT AUTO_INCREMENT PRIMARY KEY,
•     personel_id INT,
•     kayit_islem_tipi VARCHAR(50),
•     kayit_islem_tarihi TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
•     kayit_aciklama TEXT,

```

- `CONSTRAINT fk_kayit_personel FOREIGN KEY (personel_id) REFERENCES Personeller(personel_id) ON DELETE SET NULL`
- `);`

- 02_ornek_veriler.sql : Veritabanı tablolarında olması gereken test verilerini içerir.

```
USE otel_otomasyonu;

-- 1. Oda Türleri (Kat ve Cephe Planına Uygun 16 Çeşit)
INSERT INTO Oda_Turleri (odaTur_id, odaTur_adi, odaTur_kapasite,
odaTur_taban_fiyat, odaTur_aciklama) VALUES
-- Ekonomik Odalar (Arka, Deniz, Bahçe/Havuz)
(1, 'Ekonomik Oda - Arka Cephe', 1, 800.00, 'Arka cephe, temel ihtiyaçlar.'),
(2, 'Ekonomik Oda - Bahçe ve Havuz Manzaralı', 1, 1000.00, 'Giriş katında
havuza ve bahçeye bakan ekonomik oda.'),
(3, 'Ekonomik Oda - Deniz Manzaralı', 1, 1100.00, 'Üst katlarda deniz
manzaralı ekonomik oda.'),
-- Standart Tek Kişilik Odalar
(4, 'Standart Tek Kişilik - Arka Cephe', 1, 1200.00, 'Arka cephe, sessiz
standart oda.'),
(5, 'Standart Tek Kişilik - Bahçe ve Havuz Manzaralı', 1, 1400.00, 'Giriş
katında havuz gören standart tek oda.'),
(6, 'Standart Tek Kişilik - Deniz Manzaralı', 1, 1600.00, 'Deniz manzaralı
premium tek kişilik oda.'),
-- Standart Çift Kişilik Odalar
(7, 'Standart Çift Kişilik - Arka Cephe', 2, 1800.00, 'Arka cephe geniş
yataklı rahat çift kişilik oda.'),
(8, 'Standart Çift Kişilik - Bahçe ve Havuz Manzaralı', 2, 2100.00, 'Giriş
katında havuza ve bahçeye bakan çift kişilik oda.'),
(9, 'Standart Çift Kişilik - Deniz Manzaralı', 2, 2500.00, 'Kesintisiz deniz
manzaralı, lüks çift kişilik oda.'),
-- Aile Odaları
(10, 'Aile Süiti - Arka Cephe', 4, 3500.00, 'Arka cepheye bakan, 2 odalı geniş
aile süiti.'),
(11, 'Aile Süiti - Deniz Manzaralı', 4, 4500.00, 'Deniz manzaralı, geniş ve
balkonlu aile süiti.'),
-- Balayı Süitleri (8. ve 9. Katlar)
(12, 'Balayı Süiti - Arka Cephe', 2, 4000.00, 'Arka cepheye bakan standart
balayı süiti.'),
(13, 'Balayı Süiti - Jakuzili ve Deniz Manzaralı', 2, 6500.00, 'Deniz
manzaralı, odanın içinde özel jakuzisi bulunan lüks süit.'),
-- Kral Daireleri (10. Kat)
(14, 'Kral Dairesi - Arka Cephe', 2, 8000.00, 'Arka cepheye bakan geniş lüks
daire.'),
(15, 'Kral Dairesi - Deniz Manzaralı', 2, 11000.00, 'Panoramik deniz manzaralı
lüks daire.'),
(16, 'Kral Dairesi - Özel Havuzlu ve Deniz Manzaralı', 2, 15000.00, 'Kendine
ait sonsuzluk havuzu olan, deniz manzaralı VIP daire.');
```

```

-- 2. Müşteriler (15 Kişi)
INSERT INTO Musteriler (musteri_id, musteri_adi, musteri_soyadi,
musteri_tc_no, musteri_telefon, musteri_email) VALUES
(001, 'Ahmet', 'Yılmaz', '11111111111', '05321112233',
'ahmet.yilmaz@email.com'),
(002, 'Ayşe', 'Demir', '22222222222', '05552223344', 'ayse.demir@email.com'),
(003, 'Mehmet', 'Kaya', '33333333333', '05053334455',
'mehmet.kaya@email.com'),
(004, 'Fatma', 'Çelik', '44444444444', '05334445566',
'fatma.celik@email.com'),
(005, 'Ali', 'Şahin', '55555555555', '05445556677', 'ali.sahin@email.com'),
(006, 'Zeynep', 'Öztürk', '66666666666', '05556667788',
'zeynep.ozturk@email.com'),
(007, 'Mustafa', 'Aydın', '77777777777', '05327778899',
'mustafa.aydin@email.com'),
(008, 'Elif', 'Polat', '88888888888', '05058889900', 'elif.polat@email.com'),
(009, 'Osman', 'Can', '99999999999', '05339990011', 'osman.can@email.com'),
(010, 'Merve', 'Yıldız', '10101010101', '05441002233',
'merve.yildiz@email.com'),
(011, 'Emre', 'Kurt', '12121212121', '05552003344', 'emre.kurt@email.com'),
(012, 'Ceren', 'Arslan', '13131313131', '05323004455',
'ceren.arslan@email.com'),
(013, 'Can', 'Doğan', '14141414141', '05054005566', 'can.dogan@email.com'),
(014, 'Deniz', 'Tekin', '15151515151', '05335006677',
'deniz.tekin@email.com'),
(015, 'Burak', 'Şen', '16161616161', '05446007788', 'burak.sen@email.com');

-- 3. Personeller (Sadece sisteme girecek yetkililer bulunuyor.)
INSERT INTO Personeller (personel_id , personel_adi, personel_soyadi,
personel_rol, personel_kullanici_adi, personel_sifre) VALUES
(001, 'Uğur', 'Erdoğan', 'Admin', 'backend_admin_ugur', 'ugur2026'),
(002, 'Ecenur', 'Eke', 'Admin', 'frontend_admin_ece', 'ece2026'),
(003, 'Hilal', 'Çoğul', 'Admin', 'database_admin_hilal', 'hilal2026'),
(005, 'Ayten', 'Temiz', 'Temizlik', 'temizlik_ayten', 'ayten2026'),
(006, 'Kemal', 'Müdür', 'Admin', 'admin_kemal', 'kemal2026'),
(007, 'Seda', 'Gül', 'Resepsiyonist', 'res_seda', 'seda2026'),
(009, 'Zehra', 'Pırıl', 'Temizlik', 'temizlik_zehra', 'zehra2026');

-- 4. Hizmetler (10 Çeşit Ekstra)
INSERT INTO Hizmetler (hizmet_id, hizmet_adi, hizmet_birim_fiyat) VALUES
(001, 'Açık Büfe Kahvaltı', 300.00),
(002, 'SPA & Masaj', 1200.00),
(003, 'Minibar Kullanımı', 250.00),
(004, 'Havaalanı VIP Transfer', 800.00),
(005, 'Oda Servisi (Akşam Yemeği)', 600.00),
(006, 'Kuru Temizleme', 150.00),
(007, 'Ütü Hizmeti', 100.00),
(008, 'Kapalı Havuz Girişi', 200.00),

```

```

(009, 'Otopark ve Vale', 100.00),
(010, 'Geç Çıkış (Late Check-out)', 500.00);

-- 5. Odalar
INSERT INTO Odalar (oda_id, oda_no, oda_kat, odaTur_id, oda_durumu) VALUES
-- 1. KAT: Sadece Havuz ve Bahçeye Bakanlar (Ekonomik, Tek, Çift)
(1, '101', 1, 2, 'Boş'), (2, '102', 1, 2, 'Boş'), (3, '103', 1, 2, 'Boş'), (4,
'104', 1, 5, 'Boş'), (5, '105', 1, 5, 'Boş'),
(6, '106', 1, 8, 'Boş'), (7, '107', 1, 8, 'Boş'), (8, '108', 1, 8, 'Boş'), (9,
'109', 1, 5, 'Boş'), (10, '110', 1, 2, 'Boş'),
-- 2. KAT: Deniz ve Arka Cephe Karışık (Ekonomik, Standart, Aile)
(11, '201', 2, 1, 'Boş'), (12, '202', 2, 3, 'Boş'), (13, '203', 2, 4, 'Boş'),
(14, '204', 2, 6, 'Boş'), (15, '205', 2, 7, 'Boş'),
(16, '206', 2, 9, 'Boş'), (17, '207', 2, 10, 'Boş'), (18, '208', 2, 11,
'Boş'), (19, '209', 2, 1, 'Boş'), (20, '210', 2, 9, 'Boş'),
-- 3. KAT: Deniz ve Arka Cephe Karışık (Ekonomik, Standart, Aile)
(21, '301', 3, 3, 'Boş'), (22, '302', 3, 1, 'Boş'), (23, '303', 3, 6, 'Boş'),
(24, '304', 3, 4, 'Boş'), (25, '305', 3, 9, 'Boş'),
(26, '306', 3, 7, 'Boş'), (27, '307', 3, 11, 'Boş'), (28, '308', 3, 10,
'Boş'), (29, '309', 3, 9, 'Boş'), (30, '310', 3, 3, 'Boş'),
-- 4. KAT: Deniz ve Arka Cephe Karışık (Ekonomik, Standart, Aile)
(31, '401', 4, 11, 'Boş'), (32, '402', 4, 10, 'Boş'), (33, '403', 4, 1,
'Boş'), (34, '404', 4, 3, 'Boş'), (35, '405', 4, 4, 'Boş'),
(36, '406', 4, 6, 'Boş'), (37, '407', 4, 7, 'Boş'), (38, '408', 4, 9, 'Boş'),
(39, '409', 4, 11, 'Boş'), (40, '410', 4, 1, 'Boş'),
-- 5. KAT: Deniz ve Arka Cephe Karışık (Ağırlıklı Aile ve Standart)
(41, '501', 5, 11, 'Boş'), (42, '502', 5, 10, 'Boş'), (43, '503', 5, 9,
'Boş'), (44, '504', 5, 7, 'Boş'), (45, '505', 5, 6, 'Boş'),
(46, '506', 5, 4, 'Boş'), (47, '507', 5, 3, 'Boş'), (48, '508', 5, 1, 'Boş'),
(49, '509', 5, 11, 'Boş'), (50, '510', 5, 9, 'Boş'),
-- 6. KAT: Deniz ve Arka Cephe Karışık (Ağırlıklı Aile ve Standart)
(51, '601', 6, 9, 'Boş'), (52, '602', 6, 7, 'Boş'), (53, '603', 6, 11, 'Boş'),
(54, '604', 6, 10, 'Boş'), (55, '605', 6, 6, 'Boş'),
(56, '606', 6, 4, 'Boş'), (57, '607', 6, 1, 'Boş'), (58, '608', 6, 3, 'Boş'),
(59, '609', 6, 11, 'Boş'), (60, '610', 6, 7, 'Boş'),
-- 7. KAT: Deniz ve Arka Cephe Karışık (Ağırlıklı Aile ve Standart)
(61, '701', 7, 10, 'Boş'), (62, '702', 7, 11, 'Boş'), (63, '703', 7, 7,
'Boş'), (64, '704', 7, 9, 'Boş'), (65, '705', 7, 4, 'Boş'),
(66, '706', 7, 6, 'Boş'), (67, '707', 7, 1, 'Boş'), (68, '708', 7, 3, 'Boş'),
(69, '709', 7, 10, 'Boş'), (70, '710', 7, 11, 'Boş'),
-- 8. KAT: Balayı Süitleri (Arka Cephe, Deniz Manzaralı ve Jakuzili)
(71, '801', 8, 12, 'Boş'), (72, '802', 8, 12, 'Boş'), (73, '803', 8, 12,
'Boş'), (74, '804', 8, 12, 'Boş'), (75, '805', 8, 13, 'Boş'),
(76, '806', 8, 13, 'Boş'), (77, '807', 8, 13, 'Boş'), (78, '808', 8, 13,
'Boş'), (79, '809', 8, 13, 'Boş'), (80, '810', 8, 13, 'Boş'),
-- 9. KAT: Balayı Süitleri (Arka Cephe, Deniz Manzaralı ve Jakuzili)
(81, '901', 9, 12, 'Boş'), (82, '902', 9, 12, 'Boş'), (83, '903', 9, 12,
'Boş'), (84, '904', 9, 13, 'Boş'), (85, '905', 9, 13, 'Boş'),

```

```

(86, '906', 9, 13, 'Boş'), (87, '907', 9, 13, 'Boş'), (88, '908', 9, 13,
'Boş'), (89, '909', 9, 13, 'Boş'), (90, '910', 9, 13, 'Boş'),
-- 10. KAT: Kral Daireleri (Arka Cephe, Deniz Manzaralı, Havuzlu ve Deniz
Manzaralı)
(91, '1001', 10, 14, 'Boş'), (92, '1002', 10, 14, 'Boş'), (93, '1003', 10, 15,
'Boş'), (94, '1004', 10, 15, 'Boş'), (95, '1005', 10, 15, 'Boş'),
(96, '1006', 10, 15, 'Boş'), (97, '1007', 10, 16, 'Boş'), (98, '1008', 10, 16,
'Boş'), (99, '1009', 10, 16, 'Boş'), (100, '1010', 10, 16, 'Boş');

-- 6. Rezervasyonlar
INSERT INTO Rezervasyonlar (rezervasyon_id, musteri_id, oda_id,
rezerve_giris_tarihi, rezerve_cikis_tarihi, rezerve_durumu) VALUES
(1, 1, 15, '2026-05-01', '2026-05-05', 'Tamamlandı'),
(2, 2, 2, '2026-05-02', '2026-05-07', 'Tamamlandı'),
(3, 3, 45, '2026-04-28', '2026-05-04', 'Tamamlandı'),
(4, 4, 72, '2026-05-01', '2026-05-06', 'Tamamlandı'),
(5, 5, 91, '2026-04-30', '2026-05-07', 'Tamamlandı'),
(6, 6, 11, '2026-05-04', '2026-05-12', 'Onaylandı'),
(7, 7, 31, '2026-05-05', '2026-05-15', 'Onaylandı'),
(8, 8, 22, '2026-05-08', '2026-05-13', 'Onaylandı'),
(9, 9, 81, '2026-05-06', '2026-05-14', 'Onaylandı'),
(10, 10, 5, '2026-05-07', '2026-05-11', 'Onaylandı'),
(11, 11, 75, '2026-05-15', '2026-05-20', 'Beklemede'),
(12, 12, 95, '2026-05-20', '2026-05-25', 'Beklemede'),
(13, 13, 10, '2026-06-01', '2026-06-10', 'Beklemede'),
(14, 14, 55, '2026-06-15', '2026-06-20', 'Beklemede'),
(15, 15, 60, '2026-05-10', '2026-05-15', 'İptal Edildi');

-- 7. Rezervasyon Hizmetleri
INSERT INTO Rezervasyon_Hizmetleri (rezervasyon_id, hizmet_id, hizmet_adet,
hizmet_islem_tarihi) VALUES
(1, 1, 4, '2026-05-02 09:00:00'), (1, 3, 2, '2026-05-03 14:00:00'),
(2, 4, 1, '2026-05-02 10:00:00'), (2, 8, 2, '2026-05-04 11:00:00'),
(3, 5, 3, '2026-04-30 20:00:00'),
(4, 2, 1, '2026-05-02 16:00:00'), (4, 10, 1, '2026-05-06 12:00:00'),
(6, 1, 2, '2026-05-05 08:30:00'), (6, 3, 5, '2026-05-06 22:00:00'),
(7, 5, 2, '2026-05-07 19:00:00'), (9, 4, 1, '2026-05-06 09:00:00');

-- 8. Faturalar
INSERT INTO Faturalar (fatura_id, rezervasyon_id, fatura_toplam_tutar,
fatura_odeme_tarihi, fatura_odeme_yontemi) VALUES
(1, 1, 14550.00, '2026-05-05 10:00:00', 'Kredi Kartı'),
(2, 2, 9200.00, '2026-05-07 11:30:00', 'Nakit'),
(3, 3, 12400.00, '2026-05-04 09:15:00', 'Kredi Kartı'),
(4, 4, 32000.00, '2026-05-06 10:45:00', 'Kredi Kartı'),
(5, 5, 68000.00, '2026-05-07 08:30:00', 'Nakit'),
(6, 6, 18500.00, '2026-05-04 14:00:00', 'Kredi Kartı'),
(7, 9, 40800.00, '2026-05-06 12:00:00', 'Nakit');

```



```
-- 9. İşlem Kayıtları (Loglar)
INSERT INTO Islem_Kayitlari (kayit_id, personel_id, kayit_islem_tipi,
kayit_islem_tarihi, kayit_aciklama) VALUES
(1, 7, 'Rezervasyon Onayı', '2026-04-25 10:00:00', '1 numaralı Ahmet Yılmaz rezervasyonu onaylandı.'),
(2, 7, 'Giriş İşlemi', '2026-05-02 14:00:00', '2 numaralı Ayşe Demir otele giriş yaptı.'),
(3, 1, 'Sistem Ayarı', '2026-04-20 09:30:00', 'Uğur tarafından oda taban fiyatları güncellendi.'),
(4, 7, 'Rezervasyon İptali', '2026-05-07 11:15:00', '15 numaralı rezervasyon müşteri talebiyle iptal edildi.'),
(5, 2, 'Fatura Kesimi', '2026-05-05 10:30:00', '1 numaralı rezervasyonun çıkış faturası kesildi.'),
(6, 7, 'Giriş İşlemi', '2026-05-04 12:45:00', '6 numaralı (Zeynep Öztürk) rezervasyon için giriş yapıldı.'),
(7, 7, 'Çıkış İşlemi', '2026-05-07 09:10:00', '2 numaralı rezervasyon sahibi Ayşe Demir otelden ayrıldı.'),
(8, 3, 'Veritabanı Bakımı', '2026-05-08 02:00:00', 'Hilal tarafından sistem yedeklemesi ve oda optimizasyonu yapıldı.'),
(9, 6, 'Oda Durum Güncellemesi', '2026-05-06 08:00:00', '408 numaralı oda arıza nedeniyle kullanıma kapatıldı.'),
(10, 5, 'Temizlik Bildirimi', '2026-05-08 10:10:00', '205 numaralı oda temizlik personeli Ayten tarafından temizliğe alındı.');
```

- 99_tam_sifirlama.sql : Veritabanı tablolarının tamamen sıfırlanmasını sağlayan sql sorgularını içerir.

```
• USE otel_otomasyonu;
•
• SET FOREIGN_KEY_CHECKS = 0;
•
• TRUNCATE TABLE Islem_Kayitlari;
• TRUNCATE TABLE Faturalar;
• TRUNCATE TABLE Rezervasyon_Hizmetleri;
• TRUNCATE TABLE Rezervasyonlar;
• TRUNCATE TABLE Musteriler;
• TRUNCATE TABLE Odalar;
• TRUNCATE TABLE Hizmetler;
• TRUNCATE TABLE Personeller;
• TRUNCATE TABLE Oda_Turleri;
•
• SET FOREIGN_KEY_CHECKS = 1;
•
• -- Ardından sirasiyla su dosyalari tekrar calistir:
• -- 1. database/01_tablo_kurulumu.sql (sadece tablolar yoksa)
• -- 2. database/02_ornek_veriler.sql
• -- 3. database/03a_tetikleyici_oda_durumu_guncelle.sql
• -- 4. database/03b_tetikleyici_tarih_kontrolu.sql
```

- -- 5. database/04a_sanalTablo_aktif_musteriler.sql
- -- 6. database/04b_sanalTablo_fatura_bekleyenler.sql
- -- 7. database/04c_sanalTablo_vw_dashboard_ozet.sql
- -- 8. database/04d_sanalTablo_vw_rezervasyon_listesi.sql
- -- 9. database/04e_sanalTablo_vw_oda_detaylari.sql
- -- 10. database/04f_sanalTablo_vw_aktif_borclar.sql
- -- 11. database/05a_sanalMethod_fatura_hesapla.sql
- -- 12. database/05b_sanalMethod_hizmet_ekleme.sql
- -- 13. database/05c_sanalMethod_yeni_rezervasyon_ekle.sql
- -- 14. database/05d_sanalMethod_musait_oda_Ara.sql
- -- 15. database/05e_sanalMethod_rezervasyon_durum_guncelle.sql
- -- 16. database/05f_sanalMethod_oda_turu_fiyat_guncelle.sql
- -- 17. database/05g_sanalMethod_hizmet_fiyat_guncelle.sql
-

3.3.1. Tetikleyiciler (Triggers)

- 03a_tetikleyici_oda_durumu_guncelle.sql : Rezervasyon durumu değiştiğinde oda durumunun otomatik güncellenmesini sağlayan sorguyu içerir.

```
-- Rezervasyon Durumu Değiştiğinde Oda Durumunu Otomatik Güncelleyen
Tetikleyici
DROP TRIGGER IF EXISTS trg_oda_durumu_guncelle;
•
•
DELIMITER //
•
•
CREATE TRIGGER trg_oda_durumu_guncelle
• AFTER UPDATE ON Rezervasyonlar
• FOR EACH ROW
• BEGIN
•     -- 1. Aşama: Rezervasyon ONAYLANDI ise odayı DOLU yap
•     IF NEW.rezerve_durumu = 'Onaylandı' THEN
•         UPDATE Odalar
•         SET oda_durumu = 'Dolu'
•         WHERE oda_id = NEW.oda_id;
•
•     -- 2. Aşama: Rezervasyon TAMAMLANDI (müşteri çıktı) ise odayı
TEMİZLİKTE yap
•     ELSEIF NEW.rezerve_durumu = 'Tamamlandı' THEN
•         UPDATE Odalar
•         SET oda_durumu = 'Temizlikte'
•         WHERE oda_id = NEW.oda_id;
•
•     -- 3. Aşama: Rezervasyon İPTAL EDİLDİ ise odayı tekrar BOŞ yap
•     ELSEIF NEW.rezerve_durumu = 'İptal Edildi' THEN
•         UPDATE Odalar
•         SET oda_durumu = 'Boş'
•         WHERE oda_id = NEW.oda_id;
•
•
```

- `END IF;`
- `END //`
- `DELIMITER ;`

- `03b_tetikleyici_tarih_kontrolu.sql` : Rezervasyon tarih kontrolünü sağlayan sql sorgusunu içerir. Rezervasyon alımı sırasında kullanılır.

```
-- Rezervasyon Tarih Kontrolü İçin Yazılan Trigger
DROP TRIGGER IF EXISTS trg_tarih_kontrol;
DELIMITER //

CREATE TRIGGER trg_tarih_kontrol
BEFORE INSERT ON Rezervasyonlar
FOR EACH ROW
BEGIN

    -- Eğer çıkış tarihi, giriş tarihinden daha eski bir güne veya aynı
    güne:
    IF NEW.rezerve_cikis_tarihi <= NEW.rezerve_giris_tarihi THEN

        -- İşlemi iptal et ve backend tarafına şu hatayı fırlat:
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Hata: Çıkış tarihi, giriş tarihinden daha önce
veya aynı gün olamaz!';

    END IF;

END //

DELIMITER ;
```

3.3.2. Sanal Tablolar (Views)

Uygulama içinde sıkça join yapılan karmaşık sorguları tek bir sanal tabloda toplayarak performans kazanımı sağlanmıştır.

- `04a_sanalTablo_aktif_musteriler.sql` : Otelde mevcut gün içerisinde bulunan müşterilerin listelenmesini sağlayan sorguyu içerir. Bu sorgu `dashboard.html` sayfasında aktif işlemler tablosunu doldurmak için kullanılmıştır.

```
-- Otelde Aktif Bulunan Kişilerin Listesini Gösteren Sanal Tablo
•
•
• DROP VIEW IF EXISTS vw_aktif_musteriler;
• CREATE VIEW vw_aktif_musteriler AS
• SELECT
•     r.rezervasyon_id,
```

```

• m.musteri_adi,
• m.musteri_soyadi,
• m.musteri_telefon,
• o.oda_no,
• r.rezerve_giris_tarihi,
• r.rezerve_cikis_tarihi
• FROM Rezervasyonlar r
• JOIN Musteriler m ON r.musteri_id = m.musteri_id
• JOIN Odalar o ON r.oda_id = o.oda_id
• WHERE r.rezerve_durumu = 'Onaylandı';

```

- 04b_sanalTablo_fatura_bekleyenler.sql : Henüz faturası kesilmemiş müşterilerin listelenmesini sağlayan sql sorgusunu içerir. Bu sorgu işlemler.html dosyasındaki bekleyen ödemeler kısmının doldurulması için kullanılmıştır.

```

-- Henüz faturası kesilmemiş müşterilerin bilgilerini gösteren sanal tablo

DROP VIEW IF EXISTS vw_fatura_bekleyenler;
CREATE VIEW vw_fatura_bekleyenler AS
SELECT
    r.rezervasyon_id,
    m.musteri_adi,
    m.musteri_soyadi,
    o.oda_no,
    r.rezerve_cikis_tarihi
FROM Rezervasyonlar r
JOIN Musteriler m ON r.musteri_id = m.musteri_id
JOIN Odalar o ON r.oda_id = o.oda_id
LEFT JOIN Faturalar f ON r.rezervasyon_id = f.rezervasyon_id
WHERE r.rezerve_durumu = 'Tamamlandı' AND f.fatura_id IS NULL;

```

- 04c_sanalTablo_vw_dashboard_ozet.sql : Admin panelinde genel istatistiklerin bulunmasını sağlayan sql sorgusunu içerir. Bu sorgu dashboard.html sayfasında bulunan günlük operasyon özeti kartlarındaki verileri doldurmak için kullanılmıştır.

```

• -- Yönetim Paneli İçin Özet Veriler View'ı
• DROP VIEW IF EXISTS vw_dashboard_ozet;
•
• CREATE VIEW vw_dashboard_ozet AS
• SELECT
•     -- 1. Toplam Rezervasyon Kutusu
•     (SELECT COUNT(*) FROM Rezervasyonlar) AS toplam_rezervasyon,
•
•     -- 2. Beklenen Giriş Kutusu (Onay bekleyen veya girişi beklenenler)
•     (SELECT COUNT(*) FROM Rezervasyonlar WHERE rezerve_durumu =
• 'Beklemede') AS beklenen_giris_sayisi,

```

```

•
•      -- 3. Toplam Ciro Kutusu
•      (SELECT IFNULL(SUM(fatura_toplam_tutar), 0) FROM Faturalar) AS
toplaml_ciro,
•
•      -- 4. Oda Durum Özetleri
•      (SELECT COUNT(*) FROM Odalar WHERE oda_durumu = 'Dolu') AS
dolu_oda_sayisi,
•      (SELECT COUNT(*) FROM Odalar WHERE oda_durumu = 'Boş') AS
bos_oda_sayisi,
•      (SELECT COUNT(*) FROM Odalar WHERE oda_durumu = 'Temizlikte') AS
temizlikteki_oda_sayisi
• FROM DUAL;

```

- 04d_sanalTablo_vw_rezervasyon_listesi.sql : Oteldeki mevcut rezervasyonların listelenmesini sağlayan sql sorgusunu içerir. Bu sorgu admin kısmındaki rezervasyonlar sayfasının içeriğini oluşturmaktadır.
- 04e_sanalTablo_vw_oda_detaylari.sql : Odaların genel bilgilerinin listelenmesini sağlayan sql sorgusunu içerir. Bu sorgu admin kısmındaki oda durumları sayfasının içeriğini oluşturmaktadır.

```

•      -- Tüm Rezervasyon Detaylarını ve Ödeme Durumunu Gösteren View
•      DROP VIEW IF EXISTS vw_rezervasyon_listesi;
•
•      CREATE VIEW vw_rezervasyon_listesi AS
•      SELECT
•          r.rezervasyon_id,
•          m.musteri_adi,
•          m.musteri_soyadi,
•          o.oda_no,
•          ot.odaTur_adi,
•          r.rezerve_giris_tarihi,
•          r.rezerve_cikis_tarihi,
•          r.rezerve_durumu,
•          -- Fatura tablosunda kayıt varsa 'Ödendi', yoksa 'Bekliyor'
•          CASE
•              WHEN f.fatura_id IS NOT NULL THEN 'Ödendi'
•              ELSE 'Ödeme Bekliyor'
•          END AS odeme_durumu
•      FROM Rezervasyonlar r
•      JOIN Musteriler m ON r.musteri_id = m.musteri_id
•      JOIN Odalar o ON r.oda_id = o.oda_id
•      JOIN Oda_Turleri ot ON o.odaTur_id = ot.odaTur_id
•      LEFT JOIN Faturalar f ON r.rezervasyon_id = f.rezervasyon_id;

```

- 04f_sanalTablo_vw_aktif_borclar.sql : Hizmet alımları ve konaklama süreleri sonrası henüz çıkışı yapılmamış ve güncel borç kaydı bulunan müşterilerin finansal durumunu listeler. islemler.html sayfasındaki bekleyen ödemeler tablosu bu görünümünden beslenmektedir.

```

• CREATE OR REPLACE VIEW vw_aktif_borclar AS
• SELECT
•     CONCAT(am.musteri_adi, ' ', am.musteri_soyadi, ' (Oda ', am.oda_no,
•     ')) AS muster_adi,
•     (DATEDIFF(am.rezerve_cikis_tarihi, am.rezerve_giris_tarihi) *
•     od.fiyat) AS tutar
• FROM vw_aktif_musteriler am
• JOIN vw_oda_detaylari od ON am.oda_no = od.odaNumarasi
• WHERE am.rezerve_giris_tarihi <= CURDATE() AND am.rezerve_cikis_tarihi
•     >= CURDATE()
• ORDER BY tutar DESC;

```

3.3.3. Saklı Yordamlar (Stored Procedures)

- 05a_sanalMethod_fatura_hesapla.sql : Müşteriye ait rezervasyonun faturasının kesilmesi işlemi için gereken işlemleri sağlayan sql sorgusunu içerir. İşlemler ve faturalar sayfasında kullanılmıştır.

```

• DROP PROCEDURE IF EXISTS sp_FaturaKes;
• DELIMITER //
•
• CREATE PROCEDURE sp_FaturaKes(
•     IN p_rezervasyon_id INT,
•     IN p_odeme_yontemi VARCHAR(50)
• )
• BEGIN
•     DECLARE v_oda_fiyat DECIMAL(10,2);
•     DECLARE v_gun_sayisi INT;
•     DECLARE v_hizmet_toplam DECIMAL(10,2);
•     DECLARE v_genel_toplam DECIMAL(10,2);
•
•     -- 1. Oda fiyatını ve gün sayısını alalım
•     SELECT ot.odaTur_taban_fiyat, DATEDIFF(r.rezerve_cikis_tarihi,
•     r.rezerve_giris_tarihi)
•     INTO v_oda_fiyat, v_gun_sayisi
•     FROM Rezervasyonlar r
•     JOIN Odalar o ON r.oda_id = o.oda_id
•     JOIN Oda_Turleri ot ON o.odaTur_id = ot.odaTur_id
•     WHERE r.rezervasyon_id = p_rezervasyon_id;
•
•     -- 2. Hizmetlerin toplamını alalım
•     SELECT IFNULL(SUM(rh.hizmet_adet * h.hizmet_birim_fiyat), 0)

```

```

• INTO v_hizmet_toplam
• FROM Rezervasyon_Hizmetleri rh
• JOIN Hizmetler h ON rh.hizmet_id = h.hizmet_id
• WHERE rh.rezervasyon_id = p_rezervasyon_id;
•
• -- 3. Toplam hesaplama
• SET v_genel_toplam = (v_oda_fiyat * v_gun_sayisi) + v_hizmet_toplam;
•
• -- 4. Faturayı kaydedelim
• INSERT INTO Faturalar (rezervasyon_id, fatura_toplam_tutar,
fatura_odeme_tarihi, fatura_odeme_yontemi)
• VALUES (p_rezervasyon_id, v_genel_toplam, NOW(), p_odeme_yontemi);
•
• END //
• DELIMITER ;

```

- 05b_sanalMethod_hizmet_ekleme.sql: Konaklayan müşterilerin (rezervasyonların) aldıkları ekstra otel hizmetlerini (SPA, Minibar vb.) adet bilgisiyle birlikte Rezervasyon_Hizmetleri tablosuna güvenli bir şekilde işleyen saklı yordam sorgusunu içerir.

```

-- Yeni bir hizmet eklemek için basit metot
DROP PROCEDURE IF EXISTS sp_HizmetEkle;
DELIMITER //

CREATE PROCEDURE sp_HizmetEkle(
    IN p_rezervasyon_id INT,
    IN p_hizmet_id INT,
    IN p_adet INT
)
BEGIN
    INSERT INTO Rezervasyon_Hizmetleri (rezervasyon_id, hizmet_id,
hizmet_adet)
    VALUES (p_rezervasyon_id, p_hizmet_id, p_adet);
END //

DELIMITER ;

```

- 05c_sanalMethod_yeni_rezervasyon_ekle.sql : Yeni rezervasyon eklenmesini sağlayan sql sorgusunu içerir. Rezervasyon ekleme kısmında kullanılmıştır.

```

• -- Otomatik atama yerine müşterinin seçtiği oda ID'sine göre rezervasyon
yapacak şekilde prosedürü güncelledik.
• DROP PROCEDURE IF EXISTS sp_YeniRezervasyonEkle;
•
• DELIMITER //

```

```

•
• CREATE PROCEDURE sp_YeniRezervasyonEkle(
•     IN p_isim VARCHAR(50),
•     IN p_soyisim VARCHAR(50),
•     IN p_tc CHAR(11),
•     IN p_tel VARCHAR(15),
•     IN p_mail VARCHAR(100),
•     IN p_oda_id INT, -- DEĞİŞİM: Artık oda türü adı değil, doğrudan
seçilen odanın ID'si geliyor.
•     IN p_giris DATE,
•     IN p_cikis DATE
• )
• BEGIN
•     DECLARE v_musteri_id INT;
•     DECLARE v_cakisma_sayisi INT;
•
•     -- 1. ADIM: MÜŞTERİ TANIMA
•     SELECT muster_i_id INTO v_musteri_id
•     FROM Musteriler
•     WHERE muster_i_tc_no = p_tc;
•
•     IF v_musteri_id IS NULL THEN
•         INSERT INTO Musteriler (muster_i_adi, muster_i_soyadi,
muster_i_tc_no, muster_i_telefon, muster_i_email)
•         VALUES (p_isim, p_soyisim, p_tc, p_tel, p_mail);
•
•         SET v_musteri_id = LAST_INSERT_ID();
•     END IF;
•
•     -- 2. ADIM: MÜSAİTLİK SON KONTROLÜ (Çifte Rezervasyon Önlemi)
•     -- Kullanıcı ekranda odayı seçip butona basana kadar geçen sürede
başkası bu odayı almış olabilir.
•     SELECT COUNT(*) INTO v_cakisma_sayisi
•     FROM Rezervasyonlar r
•     WHERE r.oda_id = p_oda_id
•         AND r.rezerve_durumu NOT IN ('İptal Edildi')
•         AND (p_giris < r.rezerve_cikis_tarihi AND p_cikis >
r.rezerve_giris_tarihi);
•
•     -- 3. ADIM: REZERVASYON KAYDI VE LOGLAMA
•     IF v_cakisma_sayisi = 0 THEN
•         -- Çakışma yoksa rezervasyonu güvenli oluştur
•         INSERT INTO Rezervasyonlar (muster_i_id, oda_id,
rezerve_giris_tarihi, rezerve_cikis_tarihi, rezerve_durumu)
•         VALUES (v_musteri_id, p_oda_id, p_giris, p_cikis, 'Beklemede');
•
•         -- Log açıklamasını otomatik atamadan, spesifik seçime göre
güncelledik

```



```

•      INSERT INTO Islem_Kayitlari (personel_id, kayit_islem_tipi,
kayit_aciklama)
•      VALUES (1, 'Yeni Rezervasyon', CONCAT(p_tc, ' TC nolu misafir
için ', p_oda_id, ' IDli oda rezerve edildi.'));
•      ELSE
•      -- Çakışma varsa işlemi durdur ve hata fırlat
•      SIGNAL SQLSTATE '45000'
•      SET MESSAGE_TEXT = 'Üzgünüz, seçtiğiniz oda az önce rezerve
edilmiştir. Lütfen listeden başka bir oda seçiniz.';
•      END IF;
•
•      END //
•
•      DELIMITER ;

```

- 05d_sanalMethod_musait_oda_Ara.sql : Yeni rezervasyon yapılacağında boşta bulunan odaların listelenmesini sağlayan sql sorgusunu içerir. Müşteri arayüzünde rezervasyon oluşturma aşamasında kullanılmıştır.

```

-- seçilen tarihlerde musait olan odaları bulmak için oluşturulan procedure
DROP PROCEDURE IF EXISTS sp_MusaitOdaAra;
DELIMITER //

CREATE PROCEDURE sp_MusaitOdaAra(
    IN p_giris DATE,
    IN p_cikis DATE
)
BEGIN
    -- Seçilen tarihlerde çakışan rezervasyonu olmayan odaları getir
    SELECT
        v.odaNumarasi,
        v.tip,
        v.fiyat,
        v.kapasite,
        v.durumSinifi
    FROM vw_oda_detaylari v
    WHERE v.durum != 'Arızalı' -- Arızalı odaları hiç gösterme
    AND v.oda_id NOT IN (
        -- Bu tarihlerde zaten dolu olan odaların ID'lerini buluyoruz
        SELECT r.oda_id
        FROM Rezervasyonlar r
        WHERE r.rezerve_durumu NOT IN ('İptal Edildi') -- İptal edilenler
odayı meşgul etmez
        AND (
            (p_giris < r.rezerve_cikis_tarihi) AND (p_cikis >
r.rezerve_giris_tarihi)
        )
    )

```

```
);
END //

DELIMITER ;
```

- 05e_sanalMethod_rezervasyon_durum_guncelle.sql : Rezervasyonun durumunun güncellenmesini sağlayan sql sorgusunu içerir. Admin kısmında rezervasyonlar sayfasında kullanılmıştır.

```
• -- rezerve edilen odanın durumunun guncellenmesi , iptal ve check-out
işlemleri için oluşturulan procedure
• DROP PROCEDURE IF EXISTS sp_RezervasyonDurumGuncelle;
• DELIMITER //
•
• CREATE PROCEDURE sp_RezervasyonDurumGuncelle(
•     IN p_rezervasyon_id INT,
•     IN p_yeni_durum ENUM('Beklemede', 'Onaylandı', 'Tamamlandı', 'İptal
Edildi')
• )
• BEGIN
•     -- 1. Rezervasyonun durumunu güncelle
•     UPDATE Rezervasyonlar
•     SET rezerve_durumu = p_yeni_durum
•     WHERE rezervasyon_id = p_rezervasyon_id;
•
•     -- 2. İşlem kaydı tablosuna kimin ne yaptığını ekliyoruz.
•     INSERT INTO Islem_Kayitlari (personel_id, kayit_islem_tipi,
kayit_aciklama)
•     VALUES (3, 'Durum Guncelleme', CONCAT(p_rezervasyon_id, ' nolu
rezervasyon ', p_yeni_durum, ' olarak degistirildi.));
•
•     -- NOT: Burada odayı güncellemek için ekstra kod yazmıyacağım.
•     -- Elimde olan 'trg_oda_durumu_guncelle' tetikleyicimiz bunu zaten
otomatik yapıyor.
• END //
•
• DELIMITER ;
```

- 05f_sanalMethod_oda_turu_fiyat_guncelle.sql : Oda türlerinin fiyatlarının güncellenmesini sağlayan sql sorgusunu içerir. Admin kısmında ayarlar sayfasında kullanılmıştır.

```
-- Oda türlerinin taban fiyatlarını ve kapasitesini isimlerine göre dinamik
olarak güncellemek için oluşturulan prosedür
DROP PROCEDURE IF EXISTS sp_OdaTuruGuncelle;
DELIMITER //
```

```

CREATE PROCEDURE sp_OdaTuruGuncelle(
    IN p_oda_turu_adi VARCHAR(100),
    IN p_yeni_fiyat DECIMAL(10,2),
    IN p_yeni_kapasite INT
)
BEGIN
    -- Oda türünün hem fiyatını hem de kapasitesini ismine göre güncelliyoruz
    UPDATE Oda_Turleri
    SET
        odaTur_taban_fiyat = p_yeni_fiyat,
        odaTur_kapasite = p_yeni_kapasite
    WHERE odaTur_adi = p_oda_turu_adi;

END //

DELIMITER ;

```

- 05g_sanalMethod_hizmet_fiyat_guncelle.sql : Oteldeki hizmetlerin admin panelinde güncellenmesini sağlayan sql sorgusunu içerir. . Admin kısmında ayarlar sayfasında kullanılmıştır.

```

• -- Admin kısmında fiyatları güncellenen hizmetleri veritabanına
  yansıtmak için prosedür.
• DROP PROCEDURE IF EXISTS sp_HizmetFiyatGuncelle;
•
• DELIMITER //
•
• CREATE PROCEDURE sp_HizmetFiyatGuncelle(
•     IN p_hizmet_adi VARCHAR(100),
•     IN p_yeni_fiyat DECIMAL(10, 2)
• )
• BEGIN
•     -- Hizmet ismine göre fiyatı güncelle
•     UPDATE Hizmetler
•     SET hizmet_birim_fiyat = p_yeni_fiyat
•     WHERE hizmet_adi = p_hizmet_adi;
•
•     -- Kim, hangi hizmetin fiyatını değiştirdi loglayalım
•     INSERT INTO Islem_Kayitlari (personel_id, kayit_islem_tipi,
  kayit_aciklama)
•     VALUES (1, 'Hizmet Fiyat Guncelleme', CONCAT(p_hizmet_adi, ' fiyatı
  ', p_yeni_fiyat, ' TL olarak degistirildi.'));
• END //
•
• DELIMITER ;

```